Natural Language Technologies – Assignment 2025/26

Value: 50% of the module. Project submission due Sunday 14th December at 20:00. Oral assessment in January.

This is provisional and subject to Muhammad's final modifications before the 26th. .

Overview

This is the specification for the main assignment of the Natural Language Technology class.

The assignment runs as a single connected investigation. You build a corpus and characterise it, build a retrieval system and measure how well it works, then build a question-answering system over a clinical dataset and measure how and where it fails. Each part supplies material to the next, so it is worth reading the whole specification before starting.

The consistent theme is evaluation. Building these systems is now the easy part; establishing whether one works, whether it works better than something simpler, and where it breaks, is the difficult part and is where most of the marks are.

Your implementation should be made in Python running on your own local machine, using Ollama to provide LLM support. The recommendation is to run a small local instruct model — something in the 7–9B range from the Gemma, Llama or Qwen families — and to note in your report exactly which model and quantisation you used, since your results depend on it. You may call Ollama through LangChain or directly through its API, whichever you prefer. Other libraries named in individual parts below are required where they are named.

Note that you cannot use an alternative dataset to those specified in this brief.

Assessment

There are three assessment points: one project submission, one short online checkpoint during the semester, and an oral assessment in January.

Component	%	Format
Project — Parts 1 to 4, submitted together	27	Own time, one submission on 14 December
Online checkpoint	5	Individual 5-minute conversation online, w/c 24 November
Oral assessment	18	Individual 20-minute conversation in person, 12 January
Assignment total	50	

Within the project submission, marks are divided as follows: Part 1 — 4%, Part 2 — 7%, Part 3 — 8%, Part 4 — 8%.

Your oral assessment mark sets a ceiling on the marks awarded for the project. The rule is set out in full under Oral Assessment below. In short, you may use AI tools freely on the project, but you must be able to explain, modify and justify everything you submit.

Key dates

Date	What is due	Component
Sat 8 Nov	Repository, shared report document and working Ollama call committed	Part 1

Date	What is due	Component
Fri 14 Nov	PREREGISTRATION.md committed to your repository	Part 1
w/c 24 Nov	Online checkpoint — booked 5-minute slots	Checkpoint
Fri 28 Nov	Deadline to request an adjusted oral assessment format	Oral
Sun 14 Dec, 20:00	Report and repository submitted	Parts 1–4
Mon 12 Jan	Oral assessments — booked slots, in person	Oral

Task Specification

Part 1: Initial Set Up and Pre-registration — 4%

In Part 1 your task is to set up the processes for successfully performing the rest of the assignment, and to commit to a plan before you have any results.

1. Set up a shared “assignment report.docx”. The main report should be saved to your official TU Dublin OneDrive folder and shared with me as an examiner with edit access. This document will be your main report and the final version will be submitted at the end of the semester via Brightspace. It is shared so that I can see the work in progress and give you feedback on it during the semester.
2. Set up a private GitHub repository just for this assignment and share it with me. All code lives here.
3. Install Ollama to your local machine or appropriate development environment and pull a small local instruct model.
4. Configure your development environment with access to your repository, and commit code which demonstrates a successful call to an Ollama model. These gatepost steps should be completed by 8 November.
5. Commit a one-page PREREGISTRATION.md by 14 November, as set out below.

Pre-registration

Two of the four marks for Part 1 are for the pre-registration. Written before you have results, it must state:

> (i) the three document encoding and similarity approaches you will compare in Part 3, and the order in which you expect them to rank, with a sentence of reasoning for each;
>
> (ii) the two system architectures you will compare in Part 4, and which you expect to perform better, on which kinds of question;
>
> (iii) for both parts, roughly how large a difference you expect, and the rule you will use to decide whether a difference you observe is real rather than run-to-run or query-to-query noise;
>
> (iv) one sentence on what result would surprise you.

You are not marked on being right. The marks are for the quality of the reasoning behind your predictions, and later, in the report, for your account of where prediction and observation diverged and why. A pre-registration vague enough that nothing could contradict it scores poorly. A confident prediction that turns out to be wrong, well explained afterwards, scores highly.

The file is timestamped when you commit it and should not be rewritten afterwards. If your plan changes, which is normal, add a dated entry to the bottom of the file explaining what changed and why.

Part 2: Building and Characterising a Corpus — 7%

You will assemble two corpora and compare them: one that you generate with your local model, and one of real documents from the same broad domain.

The synthetic corpus

Using your Ollama model, synthesise at least 100 documents of at least 400 words each. The domain is up to you — news reports, product reviews, case notes, incident reports, short biographies — but choose one domain and stay in it, and make sure the documents contain named entities for later analysis. Save each document to an individual local file. Record the prompts, the model, the temperature and any seed you used; these belong in an appendix and they are part of the method, not a compliance exercise.

The real corpus

Assemble a comparison corpus of at least 100 real documents of comparable length in the same domain. Any of the following are acceptable sources, and you should state clearly which you used and how you obtained it:

- PubMed abstracts retrieved through the NCBI E-utilities API;
- the 20 Newsgroups or Reuters-21578 collections, available through scikit-learn and NLTK respectively;
- Dáil or Seanad debate transcripts from data.oireachtas.ie;
- public domain texts from Project Gutenberg, segmented into documents of comparable length.

If you would like to use a different real source, ask me first. Do not use a source that requires a licence you do not hold, and do not scrape a site whose terms forbid it.

Descriptive analysis

Using spaCy and other relevant text processing tools, perform a descriptive analysis of both corpora. Report, for each corpus and with the two placed side by side:

- named entities: density per thousand tokens, the distribution across entity types, and the number of distinct entities relative to the number of entity mentions;
- TF-IDF vectors for each document with respect to its own corpus, and the top terms for ten randomly selected documents from each;
- lexical diversity: type–token ratio measured over fixed-length windows, and the shape of the vocabulary growth curve as documents are added;
- sentence length distribution, not just the mean;
- near-duplicate rate within each corpus, using a method of your choice which you describe and justify.

The analysis is the deliverable, not the numbers. For every measure where the two corpora differ, say what in the generation process you think produced the difference, and what it would do to a system built on top of that corpus. Where they do not differ, say so — that is also a finding, and an interesting one.

> Your report must state at least one property of your synthetic corpus that you did not anticipate before measuring it, and what you did about it.

Part 3: Retrieval over your Corpus, Properly Evaluated — 8%

Build a Python interface that supports queries over your corpora. The interface takes a chunk of text and returns a ranked list of documents by similarity. It must implement at least three different approaches to document encoding and similarity measurement — the three you named in your pre-registration. A reasonable set would be a lexical method, a static or averaged embedding method, and a neural sentence embedding method, but the choice is yours provided the three are genuinely different in kind and you say why you chose them.

This should be done with code and tools similar to what we have looked at in class. An existing end-to-end vector database or graph database solution should not be used for this task, though you may use one elsewhere in the assignment.

Building a test collection

A ranked list is not a result until there is something to judge it against. You will therefore build a small test collection of your own:

1. Write 15 queries against your corpus. They should vary in difficulty and in kind — some answerable by keyword match, some requiring a paraphrase or a synonym to be bridged, some deliberately ambiguous.
2. For each query, pool the top 10 results returned by each of your three systems, plus the random baseline, and remove duplicates. You judge only this pool, which is what makes the exercise tractable.
3. Write down your relevance criteria before you judge anything, then judge each pooled document as relevant or not relevant. Include the criteria in an appendix.
4. Have somebody else — a classmate, a friend, anyone willing — independently judge a sample of at least 30 query-document pairs, and report the agreement between you. Name them in the acknowledgements and make sure they are happy to be named.

Pooling in this way means your measured scores are optimistic, because relevant documents that none of your systems retrieved were never judged. Say so in your report and say roughly how much you think it matters.

Evaluation

Report precision@5, mean reciprocal rank and nDCG@10 for each of your three approaches, together with:

— two mandatory baselines: a random ranking, and a lexical TF-IDF or BM25 ranking;

— the per-query spread, not only the mean across queries;

— an explicit statement, for each pair of systems you compare, of whether the difference between them is larger than the query-to-query variation, and therefore whether you are entitled to claim that one is better.

Failure analysis. Pick the three queries on which your systems disagreed most and work out why. This is worth more marks than a fourth encoding method would be.

The cross-corpus question. Run your best system over both the synthetic and the real corpus and report what changes. Relate the answer to what you measured in Part 2.

Part 4: Question Answering over Clinical Data — 8%

Here we switch from your own corpus to a small clinical dataset. You will be provided with patients.csv and dictionary.csv. patients.csv contains information on 1000 individual patients who suffer from a particular disease. All data in this dataset is synthetic. All but one of the columns can be thought of as predictive features, with the feature “survival” being a target: it measures the

number of years the patient survived. A researcher might, for example, try to build models investigating which variables predict survival.

Your task is to build a system that lets a user ask questions about this dataset in natural language, using your local model, and then to find out how well it actually works.

Build at least two architectures

Implement at least two genuinely different approaches — the two you named in your pre-registration. For example:

- — placing the data, or a summary of it, directly into the prompt context;
- — a RAG approach over row-level or group-level serialisations of the data;
- — a tool-using or agentic approach in which the model emits a query — SQL, pandas, or a constrained function call — which is executed and whose result is returned to the model.

Describe what each approach can and cannot represent about a table of 1000 rows before you report any results.

Build a gold question set

> Write 30 questions and compute the correct answer for each directly, with pandas or SQL, independently of your LLM system. This is your ground truth. The set must contain:
>
> — 10 lookup and filtering questions (“how many patients over 60 have …”);
>
> — 8 aggregation and comparison questions (“what is the mean survival for group A against group B”);
>
> — 7 multi-step questions requiring more than one operation;
>
> — 5 questions that the dataset cannot answer, because the column does not exist, the data does not support the inference, or the question is a causal claim the data cannot establish. For these, the correct behaviour is a clear refusal or a statement of what is missing.
>
> Commit the question set, the gold answers, and the code that produced them.

Evaluation

Run each architecture over the full question set at least three times, with the temperature fixed and stated. Report:

- — accuracy broken down by question type, not only overall;
- — variation across the three runs, and whether the difference between your architectures is larger than that variation;
- — the rate at which the system produced a confident answer to an unanswerable question, which is the single most important number in this part;
- — for numerical answers, how you decided what counts as correct, and what tolerance you allowed.

Failure analysis. Categorise the errors you observed and give an example of each. Fabricated values, correct method applied to the wrong column, arithmetic errors, misread dictionary definitions and plausible-sounding but unsupported causal claims are all different failures with different fixes. Say which of them you could design out, and how.

Fitness for purpose. Close this part with half a page on what would have to be true before a system of this kind could be put in front of a clinical researcher working with real patient data. Ground it in your own measured numbers rather than in general remarks about AI safety.

Reporting Requirements

These apply across Parts 2, 3 and 4 and are marked within each of them.

- **Baselines.** Every system is reported against the simplest thing that could have worked instead. A method that does not beat its baseline is a legitimate finding and is marked as such, provided you identify it and say so.
- **Variation.** Language model outputs are stochastic and small evaluation sets are noisy. Nothing is reported as a single number where a spread is available, and no difference is claimed as an improvement without a statement about whether it exceeds the noise.
- **Null results.** Your report must contain a short section listing the comparisons in which you found no effect you can defend, and why you think that is. A report in which every comparison came out as predicted will be read as a question about experimental control rather than as a strong result.
- **Reproducibility.** Model name, quantisation, temperature, any seed, library versions and hardware are stated. Somebody should be able to read your report and know what they would need to repeat it.

Online Checkpoint — 5%

In the week of 24 November you will book a five-minute individual slot online. Bring one result you have obtained so far — a corpus statistic, a retrieval score, an output from your Ollama model, anything you have actually run — share your screen, and talk me through it.

You will be asked what the result shows, what you expected, and what you intend to do next. There is nothing to prepare beyond having done some of the work, and no marks depend on the result being good or the project being on schedule. Missing the checkpoint without a documented reason scores zero for this component.

Oral Assessment — 18%

On 12 January, each student has an individual 20-minute conversation with me about their own submitted work. This takes place in person; an online alternative is available only where personal circumstances have been documented and approved. You will have your own repository and report open in front of you and may refer to them freely throughout.

Structure

The conversation follows the structure below, and sample questions of each type will be circulated on 8 November.

1. Orientation (2 min) — walk me through your repository and tell me where things are.
2. Accounting for a result (5 min) — I pick one result from your report and ask you to explain it and how confident you are that it is real.
3. Counterfactual (4 min) — what would happen if we changed one thing, and why.
4. Live micro-modification (4 min) — where in your code would a specified change go, and what else would have to change with it. You are not asked to write working code under time pressure.
5. Diagnosis (3 min) — I show you an output from one of your own runs and ask you to interpret it.
6. Limits (2 min) — what is your system bad at, and what in your evaluation could have hidden that from you.

Marks are for the substance of what you say, not the fluency of your delivery. Hesitation, thinking out loud, correcting yourself, and saying "I don't know, but here is how I would find out" all score perfectly well. There is no criterion for confidence or presentation.

The oral cap

Your band in the oral assessment sets a ceiling on the marks awarded for the project. It does not affect the marks for the checkpoint or for the oral itself.

Band	Descriptor	Cap	What this looks like
4	Full command of the submitted work	No cap	Explains design choices and why alternatives were rejected; interprets own results unprompted; answers counterfactuals about the work
3	Sound understanding, some gaps	70%	Explains what was done and why; hesitant on parts of the pipeline; recovers with prompting
2	Partial understanding	55%	Describes the work accurately but cannot account for specific choices or specific numbers; cannot answer counterfactuals
1	Familiar with the work only as a reader	40%	Can summarise the report but cannot locate anything in the code or explain any number in it
0	Cannot account for the submitted work	0%	Referred to the unfair practice process

Arrangements

- Sample questions of each type are published on 8 November.
- Slots are booked through a scheduling link circulated in December. Orals are recorded so that a second marker can moderate and so that there is a record if you wish to query your mark.
- If you have approved accommodations, or if there is anything about this format that would disadvantage you, contact me by 28 November and an adjusted format will be arranged.
- Attendance is required. Not attending without a documented and approved reason means the project cannot be awarded marks.

Submission Specification

Report

The report should be 8–12 pages in length and formatted as per the current document: standard margins, 11pt Arial font, 1.15 spacing between lines, and approximately 4pt spacing between paragraphs. Appendices sit outside this limit.

The first page is a standalone cover page and should include your name, your student number, your declaration of AI use, and the following statement:

> "I confirm that the document and related code here is all my own work, that I can explain and defend all of it, and that I did not engage in unfair practice in any way."

Do not waste space on a table of contents.

This report should be a well written, suitably structured and appropriately referenced document that justifies decisions, provides results as well as your analysis. The report should not give a background explanation of Natural Language Technology or related models, and should not simply

describe the systems that were built. Instead, write it as a narrative of your investigation, clearly showing the major steps taken, any dead ends and any learnings. The following sections should minimally be included:

- Part 1, including what you pre-registered and where prediction and outcome diverged;
- Part 2;
- Part 3;
- Part 4;
- Null results, drawn from all parts;
- AI collaboration critique (see below);
- Appendices, which do not count towards the page limit, and which should include your generation prompts, your relevance criteria and your gold question set.

Code and data

Code is submitted via the shared GitHub repository. A requirements.txt should be generated and maintained to list the packages needed to run your code. Some flexibility on style will be given so long as the marking scheme is adhered to.

The repository must also contain your synthetic corpus, the script that retrieved or assembled your real corpus, your 15 queries with their relevance judgements, and your 30 gold questions with their answers and the code that computed them. If a corpus is too large for the repository, commit the retrieval script and a manifest instead, and say where the data lives.

Conditions of submission

The following are conditions of submission rather than marked criteria. Work that does not meet them will not be marked:

- a private GitHub repository shared with me, containing all code, data or manifests, and PREREGISTRATION.md;
- a report shared with me on OneDrive with edit access, and the final version submitted via Brightspace;
- code that runs from a clean checkout following your own README, against a locally installed Ollama model;
- attendance at your scheduled oral assessment.

Use of AI Tools

You may use AI tools freely for the project. You are not required to log the prompts you used to write your code and no marks depend on doing so. The prompts you used to generate your corpus in Part 2 are a different matter — those are part of the method and must be reported.

You are required to declare, in one short paragraph on your cover page, what you used and for what — for example, “used an LLM to draft the retrieval evaluation harness, to debug a spaCy pipeline error, and to redraft section 3”.

Everything you submit is yours to defend. You may not submit code or text that you could not explain, modify or justify if asked. The oral assessment is where this is checked and its outcome takes precedence over the submitted artefacts. AI detection tools are not used and their output is not treated as evidence.

AI collaboration critique

> Include a section of approximately half a page in your report describing two specific occasions on which an AI tool gave you a suggestion that was wrong, subtly incorrect, or worse than the alternative. For each, give the suggestion, how you established that it was wrong, and what you did instead.
>
> Generic reflection on the limitations of AI scores nothing; specific, technical, verifiable instances score well. If you used these tools throughout and never caught one in an error, report that instead and say what it suggests.

Unfair Practice

Copying text or code from other sources without appropriate acknowledgement is unfair practice, including self-plagiarism, and is subject to a grade of 0 on the relevant parts of the assignment and, or, full disciplinary procedures. The checkpoint and the oral assessment are individual, closed assessments, and assistance of any kind in those is unfair practice. If you are in doubt about anything here, ask me.

For reference, here is the Students' Union guide to unfair practice: https://www.tudublinsu.ie/advice/exams/breachesofregulations/

Marking

Parts 1 to 4 are marked against the descriptors below, applied to each part separately and then subject to the oral cap. The checkpoint and the oral are marked against their own published rubrics.

Band	Descriptor
70–100%	The work is correct and the analysis is insightful. Evaluation is sound: baselines are present, variation is quantified, claims are proportionate to the evidence, and the limits of what can be concluded are stated without being asked. Failure analysis identifies mechanisms rather than listing symptoms. The report reads as an account of an investigation, including where it went wrong.
60–69%	The work is correct and competently analysed. Baselines and variation are handled properly. Analysis explains what happened but is thinner on why, and some claims run slightly ahead of the evidence.
50–59%	The work is largely complete and the results are reported accurately, but the analysis mostly describes rather than explains. Baselines may be present but under-used. Differences are claimed without much attention to whether they are real.
40–49%	The required elements are present but the treatment is mechanical. Systems are reported without baselines or without acknowledgement of variation, and the discussion does not go beyond restating the numbers.
0–39%	Elements are missing, results are not reported in a usable form, or the analysis is absent or incorrect.

Submission Details

The project is due at 20:00 on Sunday 14 December. Extensions will not be granted except in the case of documented and approved personal circumstances. Late submissions will be deducted 2 percent per day for each of the first two days late, and then 1 percent per day for each day thereafter, until a final deadline of 7 January.

All students should be available for their oral assessment on Monday 12 January. This date may be adjusted due to exam schedules in collaboration with you.